Clase 27 — Subnetting, método paso a paso

Bloque: D (Red) · **Semana:** 9 · **Duración estimada:** 120 min **Correlativa:** Clase 26 (IPv4: el datagrama y las direcciones) · **Sigue con:** Clase 28 (VLSM y planes de numeración) **Material de referencia:** Tanenbaum cap. 5 (§5.4) · RFC 1878, RFC 3021, RFC 4632, RFC 950

🎯 Al terminar esta clase vas a poder

1. Aplicar un **método de 7 pasos** para dividir cualquier bloque de direcciones, siempre en el mismo orden, con la aritmética **a la vista**.
2. Calcular los **bits de host** que necesita una subred con `2^h − 2 ≥ H`, y justificar por qué se restan **2** (red y broadcast) y cuándo se puede usar `2^h − 1`.
3. Calcular el **tamaño del prefijo**, el **número de subredes** que sale, y explicar por qué las versiones viejas de IP dividían en 2 y perdían la mitad.
4. Calcular **dirección de red, primer host, último host y broadcast** de cada subred, y verificarlos con la suma.
5. Resolver un caso de **subred cero y subred "todas unos"**, y los casos borde de `/31` **y** `/32`.
6. Explicar por qué **el broadcast de la subred cambia al hacer subnetting**, con el ejemplo de `192.168.1.0/24` dividido en cuatro `/26`.
7. Resolver el **apartado d) del Problema 2 del Examen 2011-03** entero, y justificar por qué dos subredes **de igual tamaño** no cumplen el requisito de "dejar direcciones para uso futuro".

> **Prerequisito del examen:** `2011-03 Problema 2d` (30 pts, apartado d), `2010-03 Problema 1` (renumerar tres `/24`), `2006-08 P4` (`200.40.224.0/25` en redes de 18, 11 y 9 hosts), `2010-07 P2`, `2008-12 P2`. Subnetting aparece en **prácticamente todos** los exámenes de la materia.

PARTE 1 — TEORÍA

1.1 Por qué existe el subnetting

En 1993 una empresa recibía **una clase B**: 65 536 direcciones. Con 300 computers, el 99,5 % del bloque estaba tirado. Antes de CIDR la única forma de no desperdiciar era **subnetting**, y el subnetting era **obligatorio** para las clases A, B y C: dividirse la clase en subredes para repartirlas entre routers, usando el campo de red que queda dentro de la clase.

```
Una clase B prestada  172.16.0.0/16  (65 536 direcciones)
NO se puede usar toda como una red, porque hay que poner routers
en el medio y los routers necesitan estar en subredes distintas:

┌──────────────────────────────────────────────────────────┐
│  una sola red /16  →  NO se puede poner un router en     │
│  el medio, porque los dos lados serían la misma red y    │
│  el router no tendría dos interfaces en redes distintas  │
└──────────────────────────────────────────────────────────┘

Con subnetting, el /16 se divide en subredes /24:
  172.16.0.0/24    172.16.1.0/24    172.16.2.0/24    ...
  172.16.255.0/24
  = 256 subredes de 254 hosts
```

Las dos reglas históricas que quedaron marcadas. (a) Todos los routers de la red tenían que usar **la misma máscara**: el subnetting clásico **no** adm;subredes de tamaños distintos (eso es VLSM, Clase 28). (b) La subred **cero** (la primera, con todos los bits de subred en 0) y la subred **"todas unos"** (la última) porque eran rutas por defecto legítimas en el modelo antiguo, muchas implementaciones las rechazaban. La práctica moderna **las acepta** (`RFC 950` — verificar su estado: hoy está obsoleta y su obsoletencia es justamente lo que permite el uso de la subred cero).

Como se aplica una mascara de subred, bit a bit

IP	Mascara (/26)	AND = resultado
11000000	11111111	**11000000**
10101000	11111111	**10101000**
00001010	11111111	**00001010**
00110111	11000000	**00000000**

192.168.10.55 AND 255.255.255.192 = 192.168.10.64

Lo que resulta de ese AND:

direccion de red	192.168.10.64
primer host	192.168.10.65
ultimo host	192.168.10.126
broadcast	192.168.10.127
hostables	62

El prefijo /26 significa:

26 bits de red + 6 bits de host

2^6 = 64 direcciones en la subred

64 - 2 = 62 hostables

El AND pone en 0 todos los bits de host: por eso el resultado es la direccion de RED, no una de host.

Y hoy el subnetting se usa para otra cosa. Con CIDR, una organización puede comprar un `/22` y seguir subnettingendo para todo lo demás. El subnetting no es "lo que se hacía antes de CIDR": es **la herramienta con la que se administra un bloque de direcciones**, y es exactamente lo que hay que saber para cualquier problema de plan de numeración.

1.2 El método de 7 pasos

Este es el procedimiento que hay que usar **siempre**, sin importar el enunciado. Escribilo en el papel antes de calcular nada.

```
╔═══════════════════════════════════════════════════════════╗
║  PASO 1  Bits de host:      2^h − 2 ≥ H                   ║
║  PASO 2  Tamaño del prefijo:  32 − h    →  /n             ║
║  PASO 3  N° de subredes:      2^(n − prefijo original)    ║
║  PASO 4  Hosts por subred:    2^h − 2  (verificar ≥ H)    ║
║  PASO 5  Dirección de red:    (base ÷ tamaño) × tamaño    ║
║  PASO 6  Primer y último host, y broadcast                ║
║  PASO 7  Todo a punto decimal                             ║
╚═══════════════════════════════════════════════════════════╝
```

Paso 1 — Cuántos bits de host

Se necesita `H` direcciones usables, así que el menor `h` que cumple `2^h − 2 ≥ H` es el que hay que buscar.

```
Tabla de búsqueda (la del "número mágico", RFC 1878):

h      2^h        2^h − 2      h      2^h        2^h − 2
────   ─────────  ──────────   ────   ─────────  ──────────
 1        2          0          9      512         510
 2        4          2         10     1024        1022
 3        8          6         11     2048        2046
 4       16         14         12     4096        4094
 5       32         30         13     8192        8190
 6       64         62         14    16384       16382
 7      128        126         15    32768       32766
 8      254        252         16    65536       65534
```

¿Por qué se restan 2? Porque en una subred IPv4 **dos direcciones no se pueden asignar a ningún host**:

Dirección	Quién la usa	Por qué no es asignable
Dirección de red	nadie	Identifica **la subred misma**. Si un host la tuviera, no se sabría si habla con la subred o con otro host. Con **cero** en la parte de host es la convención que define "estamos todos en la misma red".
Dirección de broadcast	nadie	Es el destino para hablar con **todos** a la vez dentro de esa subred. Si un host la tuviera, al recibir un paquete dirigido a ella creería que es para todos.

Cuándo se puede usar `2^h − 1`. Hay **un solo caso legítimo: un enlace punto a punto con exactamente dos extremos**, donde la dirección de red no se necesita para nada (nadie hace broadcast: sólo hay dos) y la dirección de broadcast **tampoco** (se habla unicast). Ahí **las dos direcciones son utilizables** y una `/31` da **2 direcciones útiles**. Eso es lo que norma la `RFC 3021` (*Using 31-Bit Prefixes on IPv4 Point-to-Point Links*, 2000, verificar):

```
Un enlace punto a punto de 2 dispositivos:

     R1 (.0)  ========================  R2 (.1)
      |             2 IPs, 2 usadas         |

Con la regla clásica: 2^1 − 2 = 0  →  ¡no hay ninguna dirección
que asignar!  Por eso la regla clásica NO alcanza y hace
falta la excepción de la RFC 3021.

Con la RFC 3021: un /31 tiene exactamente 2 direcciones y
las DOS son asignables (no hay red, no hay broadcast).

┌──────────────────────────────────────────────────────────┐
│ En un /31 NO se habla de "red" ni de "broadcast":        │
│   .0 = la dirección del primer extremo                   │
│   .1 = la dirección del segundo extremo                  │
└──────────────────────────────────────────────────────────┘
```

Y cuándo se usa `2^h − 1` en un examen que NO es punto a punto: nunca, salvo que te digan explícitamente "hay N hosts y una de ellas es el gateway ya fijo". Ahí el criterio es: si el **gateway ya tiene su dirección asignada y contada por el enunciado**, sumala a `H` antes de buscar. Es el caso del `2011-03 P2d` y lo veremos.

Paso 2 — El tamaño del prefijo

```
prefijo /n  =  32 − h

Ejemplos:   h = 7   →  /25    (128 direcciones, 126 hosts)
            h = 8   →  /24
            h = 6   →  /26    (64 direcciones, 62 hosts)
            h = 5   →  /27    (32 direcciones, 30 hosts)
            h = 4   →  /28    (16 direcciones, 14 hosts)
```

Y la **máscara decimal** sale de los `n` unos: `255.255.(2^x − 1).0` con `x = 8 − (n mod 8)`.

```
/24  →  11111111.11111111.11111111.00000000  = 255.255.255.0
/25  →  ...11111111.10000000                = 255.255.255.128
/26  →  ...11111111.11000000                = 255.255.255.192
/27  →  ...11111111.11100000                = 255.255.255.224
/28  →  ...11111111.11110000                = 255.255.255.240
/29  →  ...11111111.11111000                = 255.255.255.248
/30  →  ...11111111.11111100                = 255.255.255.252
/31  →  ...11111111.11111110                = 255.255.255.254
/32  →  ...11111111.11111111                = 255.255.255.255
```

Paso 3 — Cuántas subredes salen

Si el bloque original es `/p` y el nuevo es `/n`, se **tomaron** `n − p` **bits** de la parte que era de host, y cada uno de esos bits **se puede poner en 0 o en 1**:

```
nº de subredes  =  2^(n − p)

/24 → /27 :   n − p = 27 − 24 = 3   →  2³ =   8 subredes
/24 → /26 :   n − p = 26 − 24 = 2   →  2² =   4 subredes
/24 → /25 :   n − p = 25 − 24 = 1   →  2¹ =   2 subredes
/16 → /25 :   n − p = 25 − 16 = 9   →  2⁹ = 512 subredes
/16 → /24 :   n − p = 8              →  2⁸ = 256 subredes
```

La división en 2 de las versiones viejas de IP, y por qué se perdía la mitad. Antes de que existiera el subnetting *a la carta*, la convención era que **sólo se podía pedir un bit de subred** por vez: una clase B prestada se dividía en dos `/17`, cada uno en dos `/18`, etc. Con ese criterio:

```
Clase B  172.16.0.0/16
  1 bit de subred  →  /17  →   2 subredes de 32 766 hosts
  2 bits           →  /18  →   4 subredes de 16 382 hosts
  ...
  8 bits           →  /24  → 256 subredes de   254 hosts   ← tope clásico

El problema: con 1 bit se perdía la mitad de la clase B
(65 536 → dos subredes de 32 766 y se quedaban 2
direcciones sin poder usar, la de red y la de broadcast de
cada mitad... y además el cálculo no era múltiplo exacto).
Y, sobre todo, los routers viejos sólo comparaban UN bit de
subred, así que una subred con todos los bits de subred en 0
y la que los tenía todos en 1 se confundían con la ruta
por defecto.

La regla moderna: se pueden tomar TANTOS bits de subred como
haga falta, porque el router hace longest prefix match
(Clase 24, §1.4). Por eso hoy el "n° de subredes" es
2^(n-p) sin restricciones.
```

Paso 4 — Verificar los hosts por subred

```
hosts por subred = 2^h − 2 = 2^(32-n) − 2
chequeo:  hosts por subred  ≥  H   (el número que pedían)

Y el otro chequeo, que es el que casi nadie hace y que suma:
hosts por subred  ×  n° de subredes  ≤  2^(32-p)  −  2
(o sea: no podés haber creado subredes que no entran)
```

Paso 5 — La dirección de red de cada subred

La dirección de red de la subred *k* (contando desde 0) es el **bloque del tamaño de la subred** que empieza en el bloque base:

```
red_k = base + k × 2^(32-n)

Ejemplo con 192.168.10.0/24 dividido en /27  (tamaño 32):

  k = 0  →  192.168.10.0  + 0×32  = 192.168.10.0
  k = 1  →  192.168.10.0  + 1×32  = 192.168.10.32
  k = 2  →  192.168.10.0  + 2×32  = 192.168.10.64
  k = 3  →  192.168.10.0  + 3×32  = 192.168.10.96
  k = 4  →  192.168.10.0  + 4×32  = 192.168.10.128
  k = 5  →  192.168.10.0  + 5×32  = 192.168.10.160
  k = 6  →  192.168.10.0  + 6×32  = 192.168.10.192
  k = 7  →  192.168.10.0  + 7×32  = 192.168.10.224
```

Y el atajo mental para no equivocarse: mirá **el último octeto** de la base. Si es 0 (o múltiplo del tamaño), sumás directo en ese octeto. Si no, primero hay que bajar al múltiplo del tamaño más cercano, y **los bits que se "prestan" de los octetos de arriba son los que se encienden**. El chequeo infalible es: **la dirección de red tiene todos los bits de host en cero.**

Paso 6 — Primer host, último host y broadcast

```
primer host    =  red + 1
último host    =  red + 2^(32-n) - 2
broadcast      =  red + 2^(32-n) - 1

Y los tres chequeos que hay que hacer SIEMPRE:

  chequeo 1:  último host - primer host + 1 = n° de hosts
  chequeo 2:  broadcast = último host + 1
  chequeo 3:  (primer host - red) = 1  y  (broadcast - red) = 2^(32-n) - 1
```

Paso 7 — Todo a punto decimal

En un examen de papel es obligatorio **escribir la cuenta y el resultado en decimal**. Convertir un `/26` a `255.255.255.192` no es adivinar: es `128 + 64 = 192`. Y un broadcast como `192.168.10.191` sale de `160 + 31`.

1.3 Casos borde que se preguntan

La subred cero y la subred "todas unos"

Con los bits de subred, la primera subred es la que los tiene **todos en 0** y la última la que los tiene **todos en 1**:

```
192.168.10.0/24  en  /28  (2 bits de subred, 4 subredes)

bits de subred   subred              #  (subred cero)
──────────────   ──────────────────  ──────────────────

     00          192.168.10.0/28        0
     01          192.168.10.16/28       1
     10          192.168.10.32/28       2
     11          192.168.10.48/28       3

0 = "la subred cero"  →  ¿válida?
3 = "la subred todas unos" (los bits 11) → ¿válida?

MODELO ANTIGUO:  ambas eran PROHIBIDAS. Motivo: un router
antiguo comparaba un solo bit de subred contra la tabla, y
una entrada con todos los bits de subred en 0 se confundía
con la ruta por defecto (0.0.0.0/0); la de todos en 1 se
confundía con la dirección de broadcast "255...".

MODELO ACTUAL:    AMBAS SON VÁLIDAS. Con longest prefix
match no hay ambigüedad: /28 matchea, /0 matchea, gana /28.
`RFC 950` (Internet Subnetting, 1984) — verificar su estado
actual: la práctica moderna acepta la subred cero y la
todas-unos, y esa aceptación es justamente lo que dejó
obsoleta a la RFC 950.

EN UN EXAMEN:   si no dicen nada, dálas por válidas y
justificalo en una línea ("no hay ambigüedad con longest
prefix match"). Si el enunciado dice "no usar la subred
cero", hacé caso.
```

`/31` y `/32`

Prefijo	Tamaño	Hosts (regla clásica)	Dónde se usa
`/32`	1 dirección	1	**Una sola dirección.** Es lo que usan: la **interfaz loopback** de un router (`Router1# ip interface Loopback0 → 10.0.0.1/32`), una **ruta de host** para indicate que "este equipo en particular", y las reglas de NAT que matchean una IP exacta.
`/31`	2 direcciones	0 (por la regla clásica)	**Enlace punto a punto**, por la excepción de la `RFC 3021`: los dos extremos usan las dos direcciones. Ejemplo real: `/31` entre dos routers en un enlace de backbone.
`/30`	4 direcciones	2	El "mínimo útil" de una red con router: 2 hosts + red + broadcast.

Lo que hay que decir de `/31` en un examen: *"un `/31` tiene 2 direcciones y por la regla clásica daría 0 hosts utilizables; la `RFC 3021` legitima su uso en enlaces punto a punto, donde no hay broadcast y por lo tanto las dos direcciones son asignables. En una red con más de dos dispositivos, un `/31` es inválido."* Y la cuenta a la vista: $2^1 - 2 = 0 < 2$.

Por qué el broadcast de la subred **cambia** al hacer subnetting

Éste es **el error conceptual más común del curso**, y no es un error de cuenta: es un error de **modelo mental**.

```
ANTES de subnetting:
   192.168.1.0/24
   red = 192.168.1.0
   hosts = 192.168.1.1 ... 192.168.1.254
   BROADCAST = 192.168.1.255          ← el último de los 256

DESPUÉS de dividir en cuatro /26:
   ┌──────────────────┬──────────────────┬──────────────────┬──────────────────┐
   │  192.168.1.0/26  │ 192.168.1.64/26  │ 192.168.1.128/26 │ 192.168.1.192/26 │
   ├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
   │ red    .0        │ red    .64       │ red    .128      │ red    .192      │
   │ hosts  .1  - .62 │ hosts  .65 - .126│ hosts  .129 - .190│ hosts  .193 - .254 │
   │ BROAD  .63       │ BROAD  .127      │ BROAD  .191      │ BROAD  .255      │
   └──────────────────┴──────────────────┴──────────────────┴──────────────────┘
     .0              .63  .64              .127  .128            .191  .192            .255

   ⇒ Aparecen CUATRO broadcasts distintos: .63, .127, .191 y .255.
     El único que "ya conocías" (.255) es el de la última.
```

Por qué la respuesta conceptual es "el broadcast depende de la máscara". El broadcast de una subred es **la dirección con todos los bits de host en 1, dentro de la subred**. Con la `/24`, "todos los bits de host en 1" daba `192.168.1.255`; con la `/26`, "todos los bits de host en 1" dentro de la subred `192.168.1.0/26` da `192.168.1.63`. **El broadcast no es un número fijo de la dirección base: es una función de la máscara.** Y un host que mande `192.168.1.255` con máscara `/26` está en la subred `192.168.1.192/26`, y ese paquete es el **broadcast de su propia subred**, no un broadcast global: **nadie fuera de ese `/26` lo va a recibir nunca.**

1.4 ⚠️ Actualización a 2026

Sigue igual, de memoria — y es lo único que se pregunta:

- La fórmula `2^h − 2 ≥ H` y el **número mágico** de la `RFC 1878`.
- Red = AND; primer host = red + 1; último = red + 2^(32−n) − 2; broadcast = red + 2^(32−n) − 1.
- El broadcast depende de la máscara.
- La subred cero y la todas-unos, y el `2^(n−p)` subredes.
- `/31` para punto a punto y `/32` para loopback.

Cambió o se precisa:

- **`/31` es práctica normal desde 2010.** La `RFC 3021` (2000) está implementada en todos los routers y sistemas operativos actuales, y es **el default en muchos frameworks de configuración** (y en BGP, una `/31` se usa para el *link* local entre routers y para las sesiones `loopback`). ⇒ **Lo que era un truco de laboratorio pasó a ser la práctica de backbone.**

- **La subred cero y la todas-unos se usan sin comentarios.** `RFC 950` (1984) está **obsoleta**, y su obsoletencia es lo que hace posible el uso de ambas. Ya nadie deduplica la primera ni la última.
- **En IPv6 la pregunta es distinta y hay que saberlo.** El prefijo es de **/64 a /128** y `/64` es **la unidad de subred estándar** (`RFC 8200`): es el tamaño donde la auto-configuración de dirección (SLAAC) funciona. **En IPv6 no existen la dirección de red ni la de broadcast**: el prefijo es el identificador de la subred y **no hay broadcast** (hay **multicast** para todo). Por eso en IPv6 el cálculo es `2^(128−n) − 1` hosts, no `− 2` , y por eso `/127` y `/128` se usan como punto a punto y loopback. **Nunca apliques el `− 2` de IPv4 a IPv6.**
- **El subnetting "de manual" se reemplazó por plan de numeración con summing.** Hoy se elige el **bloque** que compra la organización (`RFC 6177` ya no recomienda `/48` por sitio final) y se subdivide en subredes de tamaños distintos (**VLSM**, Clase 28). El subnetting de igual tamaño es un ejercicio de papel; el profesional es VLSM.
- **Ya no existe "la clase que te toca".** Si te dan un `/24` y necesitás 30 hosts, no pienses "es una clase C": pensá `/27` . Decir "clase C" en 2026 es un error conceptual, aunque el examen viejo lo acepte.

Consecuencia para el examen: el método de 7 pasos se aplica **exactamente igual** en 2026. Lo que cambia son las preguntas de contexto: por qué `/31` , por qué no `− 2` en IPv6, por qué `/64` en IPv6. **Guardá esas tres.**

PARTE 2 — EJERCICIOS RESUELTOS

2.1 Ejercicio 1 — Dividir `192.168.10.0/24` en 4 subredes iguales

Datos. El bloque es `192.168.10.0/24` (256 direcciones). Se quiere dividirlo en **4 subredes iguales** y **maximizar los hosts por subred**.

Paso 1 — Bits de subred (no de host). Acá el objetivo es "4 subredes", así que conviene razonar al revés:

```
2^3 = 8      2^2 = 4  ← necesito 4
       ↑
Tomo 2 bits de la parte de host:  4 = 2²
Entonces h = 8 − 2 = 6 bits de host quedan por subred.
```

Paso 2 — El prefijo nuevo.

```
/n = 32 − h = 32 − 6 = 26      →   /26
máscara = 11111111.11111111.11111111.11000000 = 255.255.255.192
```

Paso 3 — Número de subredes.

```
2^(26 − 24) = 2² = 4   ✓  exacto, no sobra ni falta
```

Paso 4 — Hosts por subred.

```
2^6 − 2 = 64 − 2 = 62 hosts útiles por subred
2^6     = 64 direcciones por subred

Chequeo de que no me pasé:
  62 × 4 = 248 hosts útiles
  4 redes + 248 hosts + 4 broadcasts = 256  ✓  (justo)
```

Paso 5 a 7 — Las cuatro subredes completas.

```
base = 192.168.10.0     tamaño = 2⁶ = 64

┌─────┬─────────────────┬─────────┬──────────────────┬──────────┬─────────────┬──────────────────┐
│  k  │    RED          │  más    │ PRIMER HOST      │ÚLTIMO    │ BROADCAST   │   en binario     │
│     │                 │  k×64   │                  │  HOST    │             │  (4° octeto)     │
├─────┼─────────────────┼─────────┼──────────────────┼──────────┼─────────────┼──────────────────┤
│  0  │ 192.168.10.0    │ + 0     │ 192.168.10.1     │ .62      │  .63        │ 00000000..01     │
│  1  │ 192.168.10.64   │ +64     │ 192.168.10.65    │ .126     │  .127       │ 01000000..01     │
│  2  │ 192.168.10.128  │ +128    │ 192.168.10.129   │ .190     │  .191       │ 10000000..01     │
│  3  │ 192.168.10.192  │ +192    │ 192.168.10.193   │ .254     │  .255       │ 11000000..01     │
└─────┴─────────────────┴─────────┴──────────────────┴──────────┴─────────────┴──────────────────┘

Chequeo fila 1:   .126 − .65 + 1 = 62 hosts          ✓
Chequeo fila 1:   .127 = .126 + 1  (broadcast)       ✓
Chequeo fila 1:   .63 − .0 = 63 = 2⁶ − 1             ✓
Chequeo final:    .255 es el último byte y .255 − .0 + 1 = 256  ✓
```

¿Por qué `/26` y no `/27`?

```
/26  →  2^(26-24) =  4 subredes  ✓  justo lo pedido
/27  →  2^(27-24) =  8 subredes  ✗  son el doble de las pedidas
/25  →  2^(25-24) =  2 subredes  ✗  son la mitad

Con /27 haría 8 subredes de 30 hosts: sobran 4 subredes
enteras que nadie pidió. El enunciado dice "4 subredes
iguales", así que 4 = exacto, y el prefijo que da exactamente
4 es el /26. Con /26 no sobra ni falta un solo bloque.

Y si el enunciado dijera "al menos 4 subredes", entonces sí
valdría el /26... pero también habría que ver qué se pide de
cada subred (ver §2.4, que es justamente donde está la
trampa clásica del examen).
```

2.2 Ejercicio 2 — Un `/24` para redes de 100, 50 y 20 hosts

Datos. Del bloque `192.168.50.0/24` hay que sacar tres redes: una de **100 hosts**, una de **50** y una de **20**. Se busca **maximizar las direcciones libres**.

Nota anticipada: esto es **VLSM básico** (subredes de tamaños distintos), que se desarrolla entero en la Clase 28. Acá se resuelve porque es el patrón que más se repite en los exámenes.

Paso 1 — Bits de host de cada red, por separado.

```
RED DE 100 HOSTS
  2^7 - 2 = 128 - 2 = 126 ≥ 100  ✓    h = 7  →  /25  (128 dir)

RED DE 50 HOSTS
  2^6 - 2 =  64 - 2 =  62 ≥  50  ✓    h = 6  →  /26  (64 dir)

RED DE 20 HOSTS
  2^5 - 2 =  32 - 2 =  30 ≥  20  ✓    h = 5  →  /27  (32 dir)
```

Paso 2 y 3 — Prefijos y subredes.

```
/25  = 255.255.255.128      128 direcciones
/26  = 255.255.255.192       64 direcciones
/27  = 255.255.255.224       32 direcciones
```

Paso 4 — Verificación de que entra todo.

```
128 + 64 + 32 = 224 direcciones usadas de 256
256 - 224 = 32 direcciones libres = un /27 entero sobrante

Hosts: 126 + 62 + 30 = 218 hosts útiles
Redes: 3 redes + 3 gateways (1 en cada red, que ya están
       contados dentro de los H) + 3 broadcasts = 224  ✓
```

Pasos 5 a 7 — La asignación completa.

```
base = 192.168.50.0

┌──────────────┬──────────────┬──────────┬──────────────┬──────────┬──────────────┐
│ Red          │ Prefijo      │ Máscara  │ Rango de host │ Gateway  │ Broadcast    │
├──────────────┼──────────────┼──────────┼──────────────┼──────────┼──────────────┤
│ 100 hosts    │ 192.168.50.0 │ /25      │ .1   – .126  │ .127     │ .127         │
│ 50 hosts     │ 192.168.50.128│ /26     │ .129 – .190  │ .191     │ .191         │
│ 20 hosts     │ 192.168.50.192│ /27     │ .193 – .222  │ .223     │ .223         │
│ LIBRE        │ 192.168.50.224│ /27     │ (sin asignar) │ –       │ .255         │
└──────────────┴──────────────┴──────────┴──────────────┴──────────┴──────────────┘

Ojo – el "broker" (gateway) no va en su propia columna.
Lo correcto es: el gateway (.127, .191, .223) es el ÚLTIMO
host útil de cada subred.

Tabla final correcta:
┌──────────────┬──────────────┬──────────┬──────────────┬──────────────┐
│ Red          │ Prefijo      │ Máscara  │ Rango útil   │ Broadcast    │
├──────────────┼──────────────┼──────────┼──────────────┼──────────────┤
│ 100 hosts    │ 192.168.50.0 │ /25      │ .1   – .126  │ .127         │
│ 50 hosts     │ 192.168.50.128│ /26     │ .129 – .190  │ .191         │
│ 20 hosts     │ 192.168.50.192│ /27     │ .193 – .222  │ .223         │
│ LIBRE        │ 192.168.50.224│ /27     │ (sin asignar) │ .255        │
└──────────────┴──────────────┴──────────┴──────────────┴──────────────┘
```

La alineación importa. En una `/25` el tamaño es **128** (un medio del 4º octeto), así que la red tiene que empezar en `.0` o `.128` en ese octeto. En una `/26` el tamaño es 64, así que empieza en `.0, .64, .128 o .192`. En una `/27`, en `.0, .32, .64, … .224`. **Si la red que te sale no arranca en un múltiplo del tamaño, la cuenta está mal** — y esa es la verificación que hay que hacer siempre.

El detalle del último `/27` libre: `192.168.50.224/27` va de `.224` a `.255`, con **30 hosts** utilizables (`.225` a `.254`) y broadcast `.255`. Ojo: **`.255` sigue siendo un broadcast válido**, es el de esta subred, no el "broadcast de la /24". Y si en el futuro hay que crear una red de 20 hosts, esa es la que ya está separada.

2.3 Ejercicio 3 — Host concreto dentro de un `/16`

Datos. La red es **`172.16.0.0/16`** y hay que hacer una **subred `/27` que contenga al host `172.16.134.50`**. Decir la red, el rango y el broadcast.

Resolución — dos formas de hacerlo, y las dos hay que saberlas.

Forma 1: usando el número mágico. Una `/27` tiene `2⁵ − 2 = 30` hosts ⇒ sirve para `172.16.134.50` (que está entre los 30 primeros de su /24). Ahora hay que encontrar **dónde cae**.

Forma 2: con el AND, que es la que no falla.

```
Paso 1 — ¿Es una dirección de red?  El AND con la /27 debe dar
         una dirección terminada en un múltiplo de 32.

Paso 2 — El 4° octeto de la dirección es 50. En una /27 los bits
         de host del 4° octeto son los 5 últimos, así que los
         bits de red del 4° octeto son los 3 primeros:

           50 = 00110010
             └──────┘└──┘
           bits de red (3)  bits de host (5)

           Los bits de red de 50 = 001 = 1
           ⇒ el 4° octeto de la red es  00100000 = 32

Paso 3 — El AND completo:
           172.16.134.50
           172.16.134.32/27 = 172.16.134.001xxxxx
           AND          = 172.16.134.32

RED       = 172.16.134.32/27
1er host  = 172.16.134.33
último    = 172.16.134.62     (32 + 32 − 2)
broadcast = 172.16.134.63     (32 + 32 − 1)

Chequeo: .33 a .62 = 62 − 33 + 1 = 30 hosts  ✓  = 2⁵ − 2
Chequeo: .63 − .32 = 31 = 2⁵ − 1             ✓
¿Está dentro .50?  .33 ≤ .50 ≤ .62           ✓
```

El atajo, que es el que hay que tener automatizado: *en una* `/27` *(tamaño 32), la red que contiene al host con 4° octeto* `X` *es la que termina en* `X − (X mod 32)`. Para `50`: `50 mod 32 = 18`; `50 − 18 = 32`. Para `13`: `13 mod 32 = 13`; `13 − 13 = 0` ⇒ la red es `172.16.134.0/27`. Para `200`: `200 mod 32 = 8`; `200 − 8 = 192` ⇒ `192.168.1.192/27` (¡y si te equivocás de octeto, te da el broadcast de la subred anterior!).

2.4 Ejercicio 4 — "Al menos 6 subredes" en una clase C: la trampa del criterio

Datos. *Enunciado:* "Se necesita una clase C dividida en **al menos 6 subredes**. ¿Qué prefijo se debe usar?"

Paso 1 — Buscar `n` tal que `2^n ≥ 6`.

```
2° =  1    ✗
2¹ =  2    ✗
2² =  4    ✗   (4 < 6)
2³ =  8    ✓   (8 ≥ 6)   ← el menor

n = 3 bits de subred tomados
```

Paso 2 — El prefijo.

```
/n = 24 + 3 = 27        →   /27   =  255.255.255.224

Comprobación directa:
  /26 → 2² =  4 subredes   ✗  (4 < 6)
  /27 → 2³ =  8 subredes   ✓  (8 ≥ 6)
  /25 → 2¹ =  2 subredes   ✗
⇒ la respuesta es /27.
```

Paso 3 — Las ocho subredes, para que quede demostrado.

```
192.168.1.0/24  en  /27   (8 subredes de 30 hosts)

192.168.1.0/27   | 192.168.1.32/27  | 192.168.1.64/27   | 192.168.1.96/27
192.168.1.128/27 | 192.168.1.160/27 | 192.168.1.192/27  | 192.168.1.224/27

Total: 8 subredes ≥ 6 pedidas.   Hosts: 8 × 30 = 240 útiles.
```

Por qué el criterio es "al menos" y no "exactamente". Porque **6 no es una potencia de dos, y no se puede hacer exactamente 6 subredes.** El número de subredes de un bloque es **siempre** `2^(n − p)`, o sea una potencia de dos: 1, 2, 4, 8, 16… **La lista de posibilidades no tiene un 6.** Por eso el enunciado tiene que decir "**al menos** 6": obliga a **redondear hacia arriba**, y el redondeo hacia arriba de 6 es 8. **Si el enunciado dijera "exactamente 6", el problema sería imposible**, y eso mismo es la respuesta.

> ⚠️ **Aviso sobre un error frecuente en materiales de estudio.** Si en algún apunte o solución encontrás **`/26` como respuesta a este enunciado**, es un error: `/26` produce **4** subredes, que son **menos** de las 6 pedidas. La respuesta correcta para "al menos 6 subredes" es **`/27`**. El `/26` es la respuesta correcta para **"al menos 4 subredes"** (o para **"al menos 3 subredes, cada una con al menos 60 hosts"**: `n = 2` ⇒ `/26`, y `h = 32 − 26 = 6` ⇒ `2⁶ − 2 = 62 ≥ 60` ✓). **Cuando veas un número que no es potencia de dos, siempre se redondea hacia arriba y se dice por qué.**

2.5 Ejercicio 5 — Los cinco casos borde de un vistazo

Datos. Para cada prefijo, dar red, rango útil, broadcast y si es válido.

Prefijo	Tamaño `2^(32−n)`	Hosts (clásico)	Ejemplo de red	Rango útil	Broadcast	¿Dónde?
`/32`	1	`2⁰ − 2 = −1` → se trata como **1**	`10.0.0.1/32`	`.1` (sólo esa)	**no hay**	**loopback** de un router, o ruta de host
`/31`	2	`2¹ − 2 = 0`	`200.1.1.0/31`	`.0` y `.1` (**2**, por RFC 3021)	**no hay**	**enlace punto a punto**
`/30`	4	`2² − 2 = 2`	`220.10.10.28/30`	`.29`, `.30`	`.31`	red mínima **con** router
`/29`	8	`2³ − 2 = 6`	`192.168.1.8/29`	`.9` – `.14`	`.15`	red chica con router
`/24`	256	`2⁸ − 2 = 254`	`192.168.1.0/24`	`.1` – `.254`	`.255`	LAN de casa

Las notas que hay que dejar escritas:

- En `/32` no hay red ni broadcast: la dirección ES la red. Es una **ruta de host**: "este equipo en particular". Se usa para las **interfaces loopback** de los routers, y por eso los routers de BGP anuncian `/32` de loopback y no la dirección de su interfaz física: así el router es alcanzable aunque un enlace caiga.
- En `/31` (RFC 3021) las dos direcciones son asignables **sólo en punto a punto**. En una red con tres o más dispositivos sería inválido, porque necesitaría una dirección de red y una de broadcast.
- En `/30` aparecen en los exámenes en todos lados, típicamente como **enlace entre el router y el proveedor** (es el `220.10.10.28/30` del `sirc1003` y del `sirc0812`): 2 direcciones útiles y las otras dos son la red y el broadcast.

2.6 Ejercicio 6 — Un plan de numeración con tres LANs y dos enlaces

Datos. La empresa tiene `172.16.0.0/16` asignado, y necesita:

Elemento	hosts
LAN1	300
LAN2	120
LAN3	25
Enlace punto a punto R1–R2	2 (punto a punto)
Enlace punto a punto R2–R3	2 (punto a punto)

Paso 1 — Bits de host de cada uno.

```
LAN1:  300 hosts → 2^8 − 2 = 254 < 300 ✗ ; 2^9 − 2 = 510 ≥ 300 ✓  h = 9
LAN2:  120 hosts → 2^7 − 2 = 126 ≥ 120 ✓                          h = 7
LAN3:   25 hosts → 2^5 − 2 =  30 ≥  25 ✓                          h = 5
P2P:     2 puns → RFC 3021: /31                                     h = 1
```

Paso 2 — Prefijos y tamaños.

```
LAN1  h = 9  →  /23   = 512 direcciones
LAN2  h = 7  →  /25   = 128 direcciones
LAN3  h = 5  →  /27   =  32 direcciones
P2P 1       →  /31   =   2 direcciones
P2P 2       →  /31   =   2 direcciones
```

Paso 3 y 4 — Suma y verificación.

```
512 + 128 + 32 + 2 + 2 = 676 direcciones usadas

≤ 2¹⁰ = 65 536  ✓   sobra mucho (crecimiento)
```

Paso 5 a 7 — El plan completo, en orden de tamaño decreciente.

Elemento	Prefijo	Máscara	Rango útil	Broadcas
LAN1 (300 hosts)	172.16.0.0/23	255.255.254.0	.1 – .510	.511
LAN2 (120 hosts)	172.16.2.0/25	255.255.255.128	.1 – .126	.127
LAN3 (25 hosts)	172.16.2.128/27	255.255.255.224	.1 – .30	.159
Enlace P2P R1–R2	172.16.2.160/31	255.255.255.254	.161, .162	ningu
Enlace P2P R2–R3	172.16.2.162/31	255.255.255.254	.163, .164	ningu
LIBRE	172.16.3.0/22	255.255.252.0	–	.255 d

Las tres verificaciones de alineación (las que hay que hacer siempre, y son las que fallan en el examen):

```
LAN1  /23  tamaño 512 → .0      ✓ (0    es múltiplo de 512)
LAN2  /25  tamaño 128 → .2.0    ✓  (el 3er octeto 2 = 2×256; .0 del 4°
                                   es múltiplo de 128)
LAN3  /27  tamaño  32 → .2.128  ✓  (128 = 4×32, múltiplo exacto)
P2P 1 /31  tamaño   2 → .2.160  ✓  (160 = 80×2, par: obligatorio)
P2P 2 /31  tamaño   2 → .2.162  ✓  (162 = 81×2, par: obligatorio)

Si alguno no fuera múltiplo exacto, el cálculo está mal y hay
que "subir" la red al múltiplo del tamaño más cercano por
encima, desperdiciando el hueco.
```

Interpretación: **las dos redes punto a punto arrancan en dirección par**, porque un bloque de tamaño 2 sólo puede empezar en `.0/.2/.4/…` del 4º octeto. Un `/31` en `.161` sería imposible. Ese es el error típico en los planes de numeración con enlaces seriales.

2.7 Ejercicio 7 — La subnet cero y la todas-unos

Datos. Dividir `172.20.0.0/24` en subredes `/26` y decidir si la primera y la última son válidas.

```
172.20.0.0/26   | 172.20.0.64/26  | 172.20.0.128/26 | 172.20.0.192/26
red    .0       | red    .64      | red    .128     | red    .192
hosts  .1–.62   | hosts  .65–.126 | hosts  .129–.190| hosts  .193–.254
BROAD  .63      | BROAD  .127     | BROAD  .191     | BROAD  .255
────────────────┼─────────────────┼─────────────────┼────────────────
SUBNRED CERO    |                 |                 | SUBNRED TODAS UNOS
bits: 00        | bits: 01        | bits: 10        | bits: 11
```

¿Son válidas las dos extremas? Sí, en el modelo actual. Los dos bits de subred hacen que: con la subred cero, la entrada es `172.20.0.0/26`; con la `/24` hace match, con la `/16` también y con el default también, y **gana la `/26` porque es la más específica** (26 > 24 > 16 > 0). No hay ambigüedad posible. La todas-unos es `172.20.0.192/26` y gana por la misma razón.

¿Por qué estaban prohibidas? Porque los routers del modelo antiguo **no** hacían longest prefix match: comparaban bits de subred uno por uno contra la tabla, y una entrada con todos los bits de subred en 0 se confundía con la **ruta por defecto** (`0.0.0.0/0`), y la de todos en 1 con el **broadcast**. `RFC 950` (1984) reflejaba esa prohibición; hoy está obsoleta y **la prohibición también**, porque el longest prefix match la volvió innecesaria.

En un examen: dálas por válidas y escribí la línea que lo justifica ("con longest prefix match no hay ambigüedad: /26 gana sobre /24, /16 y /0"). Si el enunciado dice "sin usar la subred cero", hacé caso y empezá en la segunda: `172.20.0.64/26`.

2.8 Ejercicio 8 — Subnetting de igual tamaño contra VLSM (por qué dos subredes iguales no alcanzan)

Datos. Se tiene `100.100.100.0/24` y se necesitan **dos redes**: una de **70 hosts** y otra de **15**. Se pide "dejar la mayor cantidad posible de direcciones para uso futuro".

Resolución, calculando las dos redes por separado (que es lo correcto):

```
RED DE 70 HOSTS
  2^6 − 2 =  62 < 70  ✗    (no entra)
  2^7 − 2 = 126 ≥ 70  ✓    h = 7  →  /25 = 128 direcciones

RED DE 15 HOSTS
  2^4 − 2 =  14 < 15  ✗    (justo no entra)
  2^5 − 2 =  30 ≥ 15  ✓    h = 5  →  /27 =  32 direcciones
```

Las dos redes, con la aritmética completa:

```
RED 1  = /25  tamaño 128   base + 0×128  = 100.100.100.0/25
         red      .0
         hosts    .1    − .126
         broadcast .127

RED 2  = /27  tamaño  32   base + 1×32   = 100.100.100.128/27
         red      .128
         hosts    .129  − .158
         broadcast .159
                   ↑ alineada: 128 = 4×32  ✓
```

Cuánto queda libre:

```
usadas:  128 + 32  = 160 direcciones
libres:  256 − 160 =  96 direcciones  (un /27 y medio, o sea
                                       .160 a .255)

Quedan libres: 100.100.100.160  …  100.100.100.255
```

Por qué dos subredes de igual tamaño NO cumplirían el requisito. Esta es la parte conceptual que el enunciado de 2011-03 pide:

```
Si las dos subredes fueran iguales, las dos tienen que poder
alojar los 70 hosts:

  dos /25 → 2 × 128 = 256 direcciones   ⇒  NO queda NADA libre
              (se usa el /24 entero, 0 direcciones para el futuro)

  dos /26 → 2 ×  64 = 128 direcciones   ⇒  pero /26 sólo da 62
              hosts útiles y necesito 71 (70 + el router) ⇒ NO ENTRA

  dos /27 → 2 ×  32 =  64 direcciones   ⇒  30 < 71 ⇒ NO ENTRA

  dos /24 → 2 × 256 = 512 direcciones   ⇒  no existe un /24 libre
                                            dentro de un /24

⇒ Con subredes de igual tamaño NO HAY solución que deje
  direcciones libres. El objetivo "dejar la mayor cantidad
  posible de direcciones para uso futuro" obliga a usar
  subredes de TAMANO DISTINTO. Esa es la razón de ser del
  VLSM, y la respuesta completa al apartado d).
```

Este es el **Problema 2d del Examen 2011-03** desarrollado; la resolución completa, con el contexto del problema y la tabla de enrutamiento de R1, está en **§3.1**.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-03, Problema 2, apartado d) — dividir

`100.100.100.0/24`

> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf **Enunciado completo del Problema 2** (sólo se pide el apartado d; los demás están asignados a las clases 28, 30 y 31):
>
> > Una empresa tiene una red A (LAN Ethernet con direccionamiento privado) con 15 servidores y 70 puestos de trabajo, y se desea que puedan acceder a un servidor web en Internet con dirección IP `219.41.31.21`. Para ello el router R1 implementa NAT utilizando su dirección IP pública. Se pide: **a)** la tabla de enrutamiento de R1 y del Host H. **b)** i. los mensajes ARP con tabla ARP vacía. ii. MAC e IP de las tramas y paquetes en ambos sentidos. **c)** qué campos cambia R1 en cada sentido y su tabla NAT. **d)** *Se adquiere para la Red A el bloque de direcciones IP públicas `100.100.100.0/24`, y los administradores deciden aislar la red de servidores de los puestos de trabajo. Especifique las subredes del bloque mencionado que cumplan con este requerimiento y permitan dejar la mayor cantidad posible de direcciones IP para uso futuro. Mencione las modificaciones necesarias en el router R1 y su nueva tabla de enrutamiento.*

Solución oficial (literal del PDF):

> **d)** Subred para 70 puestos de trabajo: 70 direcciones IP + 1 para el router R1 (Además siempre se deben considerar la direcciones de red y de broadcast) → `100.100.100.0/25`. Subred para 15 servidores: 15 direcciones IP + 1 para el router R1 (Además siempre se deben considerar la direcciones de red y de broadcast) → `100.100.100.128/27`. El router R1 se conecta a cada subred y no es necesario que implemente NAT, ya que las estaciones de trabajo y los servidores tienen direcciones IP públicas. Nueva tabla de enrutamiento de R1:
>
> | Red destino / Máscara | Gateway / next hop |
> |---|---|
> | `100.100.100.0/25` | C – Directamente conectado |
> | `100.100.100.128/27` | C – Directamente conectado |
> | `220.10.10.28/30` | C – Directamente conectado |
> | `0.0.0.0/0` | `220.10.10.29` |

d-1) Por qué 70 puestos necesitan `/25`

La cuenta del método de 7 pasos, con los números del enunciado a la vista:

```
PASO 1 — bits de host

  H = 70 puestos de trabajo
  + 1 para la interfaz de R1 dentro de esta subred
  ──────────────────────────────────
  H = 71 direcciones que hay que poder asignar

  2^6 − 2 =  64 − 2 =  62  < 71   ✗   NO ENTRA
  2^7 − 2 = 128 − 2 = 126  ≥ 71   ✓   h = 7

PASO 2 — prefijo

  /n = 32 − h = 32 − 7 = 25    →   /25
  máscara = 11111111.11111111.11111111.10000000 = 255.255.255.128

PASO 3 — subredes
  No aplica: acá se pide UNA subred de un tamaño dado.

PASO 4 — verificación
  hosts útiles = 2⁷ − 2 = 126 ≥ 71  ✓   (sobran 55)

PASO 5 — dirección de red
  base = 100.100.100.0
  La primera /25 de un /24 arranca SIEMPRE en .0 (128 = 2×64, y el
  bloque 0 es el primero).  Y .0 es múltiplo de 128  ✓
  RED = 100.100.100.0/25

PASO 6 — host, host y broadcast
  1er host   = 100.100.100.0  + 1     = 100.100.100.1
  último host= 100.100.100.0  + 128 − 2 = 100.100.100.126
  broadcast  = 100.100.100.0  + 128 − 1 = 100.100.100.127

PASO 7 — decimal  (ya está en decimal; la cuenta binaria sería)
  128 = 2⁷   ⇒  en el 4° octeto, bit 7 = 128
  .127 = 01111111   .126 = 01111110   .1 = 00000001   .0 = 00000000
```

Por qué hace falta "el +1 del router". R1 tiene que tener **una interfaz dentro de cada subred**, y esa dirección también sale del bloque de la subred. Los 70 puestos más la interfaz de R1 son 71 direcciones asignables. La oficial lo dice explícitamente ("70 direcciones IP + 1 para el router R1"), y es el detalle que hace que la cuenta dé bien: si se olvidara el `+1`, el cálculo sería `2⁶ − 2 = 62 ≥ 70`, que es falso, y uno caería en un `/26` que **no entra**.

Detalle no exigido por el enunciado pero que hay que dejar dicho: la dirección de red y la de broadcast también se consumen (de ahí "Además siempre se deben considerar las direcciones de red y de broadcast"). Son las dos que justifica el `− 2`.

d-2) Por qué 15 servidores necesitan `/27`

```
PASO 1 — bits de host

  H = 15 servidores
  + 1 para la interfaz de R1
  ──────────────────────────
  H = 16

  2^4 − 2 = 16 − 2 = 14 < 16   ✗   ¡justo no entra!
  2^5 − 2 = 32 − 2 = 30 ≥ 16   ✓   h = 5

PASO 2 — prefijo
  /n = 32 − 5 = 27    →   /27   = 255.255.255.224

PASO 4 — hosts útiles = 2⁵ − 2 = 30 ≥ 16  ✓   (sobran 14)

PASO 5 — DIRECCIÓN DE RED: aquí está la parte delicada.
  Tamaño de la /27 = 2⁵ = 32.
  Las /27 dentro de 100.100.100.0/24 arrancan en el 4° octeto
  en los múltiplos de 32:

     0, 32, 64, 96, 128, 160, 192, 224

  Como la /25 anterior ocupa .0 a .127, la siguiente /27 tiene
  que arrancar en 128 (o en un múltiplo de 32 mayor):
     128 = 4 × 32  ✓  alineada

  RED = 100.100.100.128/27

PASO 6 —
  1er host    = 100.100.100.128 + 1  = 100.100.100.129
  último host = 100.100.100.128 + 30 = 100.100.100.158
  broadcast   = 100.100.100.128 + 31 = 100.100.100.159

PASO 7 — verificación
  .159 − .129 + 1 = 31 − 1 + 1 = 31... no: 158 − 129 + 1 = 30 hosts ✓
  .159 − .128 = 31 = 2⁵ − 1  ✓
  .159 es múltiplo de 32 menos 1  ✓
```

Lo que hay que mirar con atención: el `/28` habría sido "justo" por un host. `2⁴ − 2 = 14` y son 15 servidores + 1 router = 16. **Una sola dirección de diferencia** y la respuesta salta de `/28` a `/27`. Si en un examen te encontrás dudando entre dos prefijos, **escribí las dos cuentas completas y dejá que se vean los números**; el error salta a la vista.

d-3) Por qué NO dos subredes de igual tamaño

Este es el punto conceptual que vale la mayor parte del apartado, y la oficial lo resuelve dando las dos subredas de tamaños distintos sin explicarlo. La explicación completa:

```
El requisito es triple:
  (1) aislar la red de servidores de la de puestos de trabajo
  (2) que 70 puestos + R1 quepan
  (3) que 15 servidores + R1 quepan
  (4) DEJAR LA MAYOR CANTIDAD POSIBLE DE DIRECCIONES PARA EL FUTURO

Si las dos subredes fueran del MISMO tamaño (subnetting de
igual tamaño, que es lo que se enseñaba en 1994), las dos
tendrían que poder alojar los 70 puestos + R1 = 71:

  dos /25 → 2 × 128 = 256   → usa el /24 ENTERO, quedan 0 libres
  dos /26 → 2 ×  64 = 128   → pero 62 < 71: NO ENTRA
  dos /27 → 2 ×  32 =  64   → pero 30 < 71: NO ENTRA
  dos /24 → 2 × 256 = 512   → no existe un /24 libre dentro del /24

⇒ con subredes iguales NO HAY solución que deje direcciones
  libres. El requisito (4) obliga a subredes de TAMAÑO
  DISTINTO: /25 para los puestos y /27 para los servidores.

Eso es, literalmente, VLSM (Variable Length Subnet Masks), y
por eso este apartado es el puente entre el subnetting de
igual tamaño y la Clase 28.
```

Cuánto queda libre, y por qué es el máximo posible:

```
usadas:  128 (/25) + 32 (/27) = 160
libres:  256 − 160 = 96 direcciones

Que es el máximo, y la demostración es por bajar de tamaño:
  /24 para puestos + /26 para servidores = 256 + 64 = 320 > 256 ✗
  /25 para puestos + /28 para servidores = 128 + 16 = 144, pero
      /28 sólo da 14 hosts y necesito 16  ✗
  /25 + /27                           = 160  ✓  ← el único que entra

Los 96 libres quedan en 100.100.100.160 – 100.100.100.255,
que es lo que queda disponible para el crecimiento futuro.
```

d-4) Las modificaciones en R1 y su nueva tabla

Modificaciones necesarias en el router R1:

1. **Agregar una segunda interfaz física** (o un puerto de un switch) para conectar la **red de servidores**, porque antes toda la red A era **una sola** LAN y ahora son **dos**. R1 pasa de 1 interfaz LAN a **2**.
2. **Configurar en cada interfaz la dirección de su subred:** una en `100.100.100.x/25` (red de puestos) y otra en `100.100.100.y/27` (red de servidores), con su máscara correspondiente. **Las dos subredes pueden estar conectadas al mismo switch** si el switch es capa 2 y las dos redes están en VLANs distintas, o a puertos distintos de un router con dos interfaces.
3. **Sacar la tabla NAT.** La oficial lo dice: "*no es necesario que implemente NAT, ya que las estaciones de trabajo y los servidores tienen direcciones IP públicas*". Antes R1 traducía `192.168.1.x` (privadas) a su IP pública; ahora **todas las direcciones del bloque son públicas**, así que **el paquete pasa sin traducción** y la tabla NAT desaparece.
4. **Ajustar la ruta por defecto** si cambia la salida a Internet (en el enunciado la salida sigue siendo la interfaz `220.10.10.28/30`, así que la ruta por defecto **no cambia**).

Nueva tabla de enrutamiento de R1 (solución oficial, con mi lectura de por qué es así):

```
┌──────────────────────────────────────┬──────────────────────────┐
│ Red destino / Máscara                │ Gateway / next hop       │
├──────────────────────────────────────┼──────────────────────────┤
│ 100.100.100.0/25                     │ C – Directamente conn.   │
│ 100.100.100.128/27                   │ C – Directamente conn.   │
│ 220.10.10.28/30                      │ C – Directamente conn.   │
│ 0.0.0.0/0                            │ 220.10.10.29             │
└──────────────────────────────────────┴──────────────────────────┘

Lectura:
• Las dos primeras líneas son las DOS subredes nuevas, cada
  una directamente conectada (no hay next hop: R1 tiene una
  interfaz física en cada una).
• La tercera es el enlace con el ISP, que ya existía.
• La cuarta es la ruta por defecto hacia Internet, que NO
  cambia: el gateway sigue siendo 220.10.10.29 (la interfaz
  del router del ISP en esa /30).
• La línea 192.168.1.0/24 que estaba antes YA NO EXISTE:
  se reemplazó por la /25 de puestos. Esa es "la modificación
  necesaria en la tabla".
```

Nota mía sobre la solución oficial, y una diferencia que vale la pena discutir. La oficial **no dice qué pasa con la tabla ARP ni con el filtrado entre las dos redes**. El enunciado dice "aislar la red de servidores de los puestos de trabajo", y eso tiene una lectura más fuerte que "dos subredes distintas": **que los puestos no alcancen a los servidores y viceversa**. Con la tabla de enrutamiento sola **no se aísla nada**: R1 sabe cómo ir a las dos redes y **reenvía**, así que un puesto podría hacer ping al servidor si R1 tiene *IP forwarding* activado.

Mi lectura: para aislar de verdad, o bien (a) las dos redes están en **VLANs distintas** de un switch y hay ACLs o una política de seguridad que impide el tráfico entre ellas, o bien (b) se configura en R1 una **lista de acceso que bloquee el tráfico entre `100.100.100.128/27` y `100.100.100.0/25`**, o bien (c) se entiende "aislar" en el sentido **topológico**: que sean subredes distintas con dominios de broadcast distintos, que es lo que implica la tabla de enrutamiento.

Mi respuesta al apartado d) sería: *dos subredes de tamaños distintos, `100.100.100.0/25` para los 70 puestos + R1 (126 hosts útiles) y `100.100.100.128/27` para los 15 servidores + R1 (30 hosts útiles), dejando 96 direcciones libres; R1 necesita una segunda interfaz LAN, se le configuran las direcciones de cada subred con su máscara, se elimina la tabla NAT porque ahora todas las direcciones son públicas, se reemplaza la entrada `192.168.1.0/24` por las dos nuevas en la tabla de enrutamiento, y — si "aislar" se entiende como impedir el tráfico entre las redes — hace falta además una ACL o VLAN que separe ambas.*

> **Tope del apartado (los 30 pts del problema se reparten entre los cuatro apartados):** las dos subredes con sus cuatro valores (red, primer host, último host, broadcast) y los dos prefijos correctos; **la justificación de por qué son de tamaños distintos** (porque con iguales no queda nada libre); el cálculo de cuántas direcciones quedan libres; la interfaz adicional y el **sacar el NAT**; y la tabla nueva con las tres líneas directas y el default.

3.2 Nota de coordinación: el resto del Problema 2 de 2011-03

Los otros tres apartados del mismo problema están asignados a otras clases y **no se resuelven acá**, pero conviene saber dónde están:

Apartado	Qué es	Clase
a)	tabla de enrutamiento de R1 y del host H (con `192.168.1.0/24` y `0.0.0.0/0`)	**30** — Tablas de enrutamiento, next hop y ARP
b)	mensajes ARP con la tabla ARP vacía, y MAC/IP de tramas y paquetes en ambos sentidos	**30** — Tablas de enrutamiento, next hop y ARP
c)	qué campos cambia R1 en cada sentido (IP origen + puerto) y la tabla NAT	**31** — NAT, PAT y sus tricks
d)	las dos subredes del `/24` comprado	**esta clase**

El orden de las clases sigue el del problema, así que si estás preparando ese examen, el orden de estudio es 30 → 31 → 27. Y el enunciado completo del problema está al principio de §3.1, así que lo tenés a mano.

PARTE 4 — CIERRE

📌 Resumen

1. El **método de 7 pasos** es lo que hay que usar siempre: `2^h − 2 ≥ H` → `/n = 32 − h` → `2^(n−p)` subredes → verificar `2^h − 2 ≥ H` → `red = base + k × 2^(32−n)` → `+1`, `−2`, `−1` → decimal.
2. Se restan **2** porque la **dirección de red** identifica la subred y la de **broadcast** habla con todos. En **punto a punto** (`/31`, `RFC 3021`) **no hace falta ninguna de las dos** y las 2 direcciones son útiles.
3. **Nº de subredes =** `2^(n − p)`, sin la restricción antigua de un bit por vez. Dividir en 2 era elSubnetting clásico de antes de CIDR.
4. Los tres valores se verifican siempre: `último − primero + 1 = hosts`, `broadcast = último + 1`, `broadcast − red = 2^(32−n) − 1`.
5. La **dirección de red tiene todos los bits de host en 0**: ése es el chequeo infalible, y **tiene que empezar en un múltiplo exacto del tamaño** de la subred.
6. **El broadcast depende de la máscara.** `192.168.1.0/24` en cuatro `/26` da **cuatro** broadcasts: `.63`, `.127`, `.191`, `.255`.
7. **La subred cero y la todas-unos son válidas hoy**: con *longest prefix match* no hay ambigüedad. `RFC 950` está obsoleta.
8. `/32` = loopback o ruta de host · `/31` = punto a punto · `/30` = red mínima con router.
9. **"Al menos 6 subredes"** ⇒ `/27`, porque el número de subredes es siempre una potencia de dos y hay que **redondear hacia arriba**. `/26` daría 4 subredes, menos de 6.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué la fórmula es `2^h − 2` y no `2^h`?

▶ Respuesta

2. ¿Cuándo se puede usar `2^h − 1` en vez de `2^h − 2`? Poné un ejemplo y el número.

▶ Respuesta

3. Dividí `10.0.0.0/16` en `/26`. ¿Cuántas subredes salen y cuál es la dirección de red de la subred número 5?

▶ Respuesta

4. ¿Por qué el broadcast de una subred no es "el último número de la dirección base"?

▶ Respuesta

5. "Se necesitan al menos 6 subredes de una clase C." ¿Qué prefijo, y por qué?

▶ Respuesta

6. ¿La subred cero y la subred "todas unos" son válidas? ¿Por qué antes no lo eran?

▶ Respuesta

7. Un enlace entre dos routers tiene la subred `192.168.5.4/30`. ¿Cuáles son la red, el gateway, el host útil y el broadcast? ¿Y si en vez fuera `/31`?

▶ Respuesta

8. En el `2011-03 P2d`: por qué 70 puestos necesitan `/25` y por qué dos subredes iguales no cumplen el requisito?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1103` / `sirc1103`** — Problema 2 (30 pts), apartado **d**: dividir `100.100.100.0/24` para 70 puestos + 15 servidores maximizando las direcciones futuras. El problema de subnetting más importante del banco.
- **Examen `eirc1003` / `sirc1003`** — Problema 1: tres `/24` que no se pueden renumerar agrupar sin tocar una red. **Examen `eirc0912` / `sirc0912`** — Problema 1: `192.168.108.0/24` + `192.168.109.0/24` sí se agregan en una `/23`.
- **Examen `eirc0608` / `sirc0608`** — P4: `200.40.224.0/25` subdividido en redes de 18, 11 y 9 hosts (VLSM). · **Examen `eirc0812` / `sirc0812`** — P2: plan de numeración + NAT + tunneling.
- **Examen `eirc1007` / `sirc1007`** — P2: subredes LAN2 y LAN3 con 250 hosts desde `192.168.2.0/23`.
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red" — §5.4 el protocolo IP: la cabecera, la máscara y las subredes.
- **RFC 1878** — Requirements for IP Address Reservation (1996): **el anexo con el "número mágico"**, la tabla de hosts por prefijo que hay que usar en el papel.
- **RFC 3021** — Using 31-Bit Prefixes on IPv4 Point-to-Point Links (2000): la excepción del `/31`. ⚠️ No figura en `ACTUALIZACIONES.md`: verificá el número antes de citarlo.
- **RFC 4632** — CIDR (2006): el prefijo de longitud libre que hace posible todo esto. · **RFC 950** — Internet Subnetting (1984): el modelo viejo y la prohibición de la subred cero; **obsoleta**, verificá su estado.
- **RFC 8200** — IPv6 (2017): el prefijo `/64` como unidad de subred, y por qué en IPv6 **no hay dirección de red ni broadcast** (el cálculo es `2^(128-n) - 1`).
- **RFC 6177** — ya no recomienda asignar `/48` por sitio final; el plan de numeración moderno se elige por bloque y se subdivide con VLSM.
- **Clase siguiente:** 28 — VLSM y planes de numeración